

EMBEDDING ATLAS: Low-Friction, Interactive Embedding Visualization

Donghao Ren^{*†}
Apple

Fred Hohman^{*‡}
Apple

Halden Lin[§]
Apple

Dominik Moritz[¶]
Apple

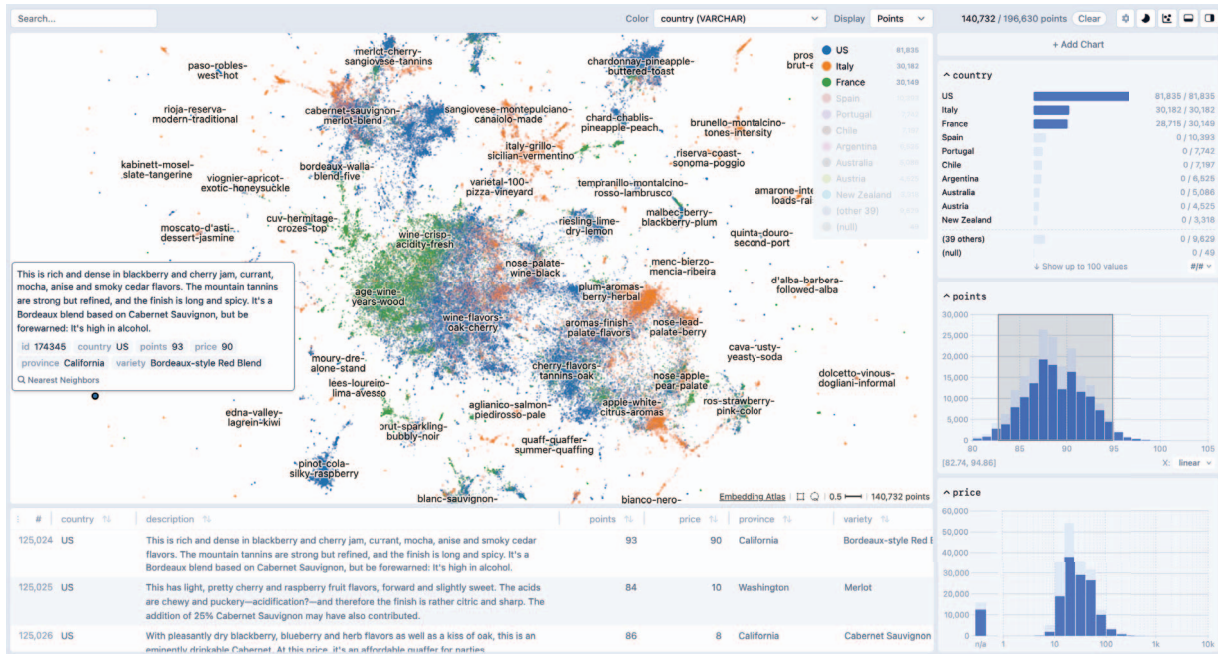


Figure 1: The EMBEDDING ATLAS user interface visualizing the embedding of 196,630 wine reviews. A user has filtered the embedding to only show wines from the US, France, and Italy with “points” in a specific range, and selected one outlier from the embedding to read its review and inspect its metadata. Try the demo at: <https://apple.github.io/embedding-atlas>

ABSTRACT

Embedding projections are popular for visualizing large datasets and models. However, people often encounter “friction” when using embedding visualization tools: (1) barriers to adoption, e.g., tedious data wrangling and loading, scalability limits, no integration of results into existing workflows, and (2) limitations in possible analyses, without integration with external tools to additionally show coordinated views of metadata. In this paper, we present EMBEDDING ATLAS, a scalable, interactive visualization tool designed to make interacting with large embeddings as easy as possible. EMBEDDING ATLAS uses modern web technologies and advanced algorithms—including density-based clustering, and automated labeling—to provide a fast and rich data analysis experience at scale. We evaluate EMBEDDING ATLAS with a competitive analysis against other popular embedding tools, showing that EMBEDDING ATLAS’s feature set specifically helps reduce friction, and report a benchmark on its real-time rendering performance with millions of points. EMBEDDING ATLAS is available as open source to support future work in embedding-based analysis.

Index Terms: Embedding visualization, visual analytics.

^{*}Authors contributed equally.

[†]e-mail: donghao@apple.com

[‡]e-mail: fredhohman@apple.com

[§]e-mail: halden@apple.com

[¶]e-mail: domoritz@apple.com

1 INTRODUCTION

Effectively visualizing and summarizing large-scale, unstructured datasets remains a persistent challenge for developing machine learning (ML) models. One popular technique is to embed data (e.g., text, images) into high-dimensional (yet uninterpretable) vectors, and then project the vectors into 2D using dimensionality reduction techniques (e.g., PCA [11], t-SNE [25], or UMAP [15]) for visualization. Projections may preserve local neighborhoods and global structure to ultimately make sense of the original data. These *embedding visualizations* have recently seen a resurgence [26, 17, 12, 1, 7]. Although static embedding visualizations are common, interactive tools enhance exploration and reveal deeper insights through user-driven analysis.

Like any data analysis tool, the usability of embedding visualizations hinge upon their “utility-to-tedium” ratio. The best embedding tools make it easy to get data in, do work, and get insight and resulting data out, meshing seamlessly into existing development workflows. Every bit of friction encountered by a user during installation, set up, and use, lowers the chance of them engaging with the tool and their data. Other examples of friction include if loading data is cumbersome or requires extra preprocessing, sluggish performance as data size increases, or if one cannot cross-filter and examine other metadata alongside the embedding. Existing tools today can be a dead end for people’s analyses.

In this paper, we introduce EMBEDDING ATLAS, a scalable, interactive embedding visualization tool designed to make exploring large embeddings as easy as possible. A core design principle of EMBEDDING ATLAS is to minimize *friction*, focusing people’s time on looking at their data rather than configuring the tool. EMBEDDING ATLAS builds upon the latest web technologies [19, 10] and

visualization research [5] for browser-based data analysis, implements recent algorithms for automatic data clustering and labeling [22], and scales to millions of points while keeping smoother interactivity performance than previous work. We detail EMBEDDING ATLAS's features compared to existing tools, and evaluate its performance on embedding rendering.

Our contributions include:

- **EMBEDDING ATLAS**: a scalable, interactive, low-friction embedding visualization tool.
- **A competitive analysis** of EMBEDDING ATLAS against other popular embedding visualizers, and a **benchmark** of its rendering performance.
- **An open-source implementation** of the tool, available at <https://apple.github.io/embedding-atlas>.

2 RELATED WORK

One of the most widely adopted tools for embedding visualization is the TensorBoard Embedding Projector [24], popularized by its integration into TensorFlow. The Embedding Projector allows users to show the results of dimensionality reduction (e.g., including UMAP [15], t-SNE [25], and PCA [11]), load model checkpoints, attach metadata, and navigate an embedding by zooming, panning, and inspecting neighborhoods on the fly. More recently, tools like WizMap [26] have pushed the frontier of embedding visualization by targeting scalability and performance. WizMap is designed to handle millions of points in-browser using WebGL and Web Workers. By using a novel multi-resolution summarization technique along with a map-like interaction paradigm, WizMap allows users to smoothly navigate embeddings. In the commercial space, Nomic Atlas [17] is a cloud-based platform for scalable, interactive embedding visualization that leverages Nomic's embedding models to provide full text and vector search. Another tool, DataMapPlot [14], is designed to help users create embedding plots for use in presentations, posters, and papers. It focuses on producing static plots or lightly interactive plots that are appealing with minimal effort. Lastly, Latent Scope [9] leverages sparse-autoencoders [18, 16, 20] for better search and clustering to support ML interpretability [3]. We compare EMBEDDING ATLAS's features with these existing tools in Section 4.

Beyond dedicated and established tools, embedding visualizations also appear in task-specific visual analytics systems that handle unstructured data [6]. These systems are often accompanied with filters, charts, or tables to support specific analytic goals. EMBEDDING ATLAS, as a general-purpose embedding tool, aims to cover a significant portion of the design space of such systems.

EMBEDDING ATLAS integrates web-based interactive embedding visualization with dynamic charts and data tables. Through Mosaic [5], it enables smooth cross-filtering across all of its components. We strive to deliver a streamlined analytical experience with minimal setup overhead, allowing even novice users to start exploring their data quickly. At the same time, advanced users can seamlessly embed the system into their analytical applications.

3 EMBEDDING ATLAS FEATURES

3.1 Motivating Example

Before introducing the features of EMBEDDING ATLAS, we begin with an overview and a motivating usage example. Imagine ROBIN, a data analyst, who receives a new dataset containing ~200k wine reviews. She starts by loading a Parquet file into EMBEDDING ATLAS by simply dragging and dropping the file into the browser, and selects the wine description column to generate embeddings. Without any data preprocessing, EMBEDDING ATLAS automatically generates high-dimensional vectors, projects them into 2D, and computes clusters and labels automatically in the browser.

The initial view presents an embedding visualization with cluster labels for easier exploration, along with charts displaying the distri-

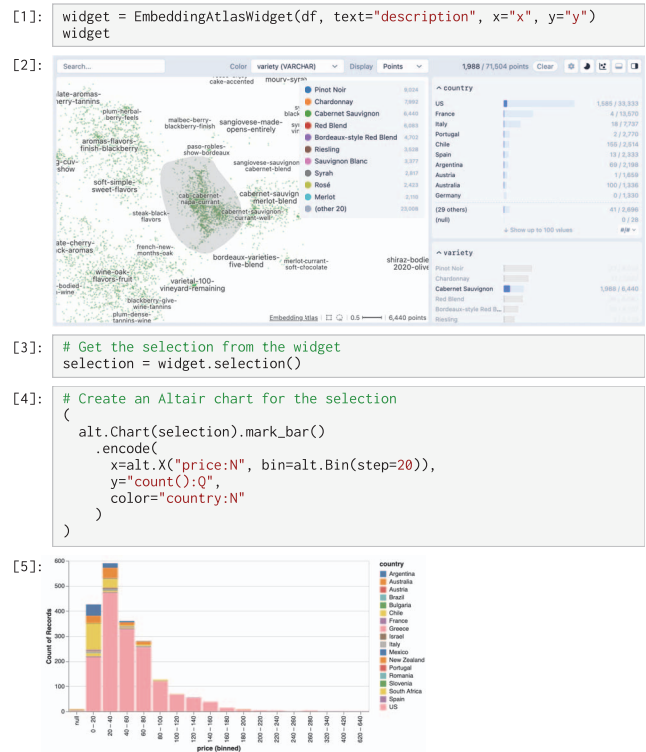


Figure 2: EMBEDDING ATLAS widget in Jupyter Lab.

butions of all metadata columns (as shown in Figure 1). From looking at the country metadata chart, ROBIN notices that the dataset is skewed: most of the wines are from the United States. She then colors the embedding points by country to see the distribution of reviews across regions. Because there are noticeable clusters by region, she hypothesizes that wines from the same region may taste more similar to each other than to those from other regions.

Next, she wants to find the best wines for the cheapest price. To find these wines, she analyzes the correlation between wine price and review score points by creating a 2D heatmap. The heatmap reveals a positive correlation and allows her to interactively select reviews with high scores but relatively low prices, discovering that French wines are heavily represented. She exports this subset to a Parquet file to save for the future. While ROBIN performed this analysis directly in the web browser, she alternatively could have started in a Jupyter notebook using the EMBEDDING ATLAS widget to carry out the same exploration. In the Jupyter workflow, her final selection can be exported as a DataFrame object for continued analysis within the notebook environment (Figure 2 shows another example of using EMBEDDING ATLAS in a notebook, where we graph the resulting selection with an Altair chart).

3.2 System Features

When designing EMBEDDING ATLAS, we wanted a tool that was easy to use and performant. As regular embedding tool users ourselves, we reviewed existing popular tools (see Section 4) to determine features that EMBEDDING ATLAS needed to cover, features that could be improved, and new features altogether. Below, each of EMBEDDING ATLAS's features (F1–F8) are first described, then detail how they reduce friction to help users explore, understand, and interpret large-scale embeddings and their metadata.

F1. Fast interactive rendering of massive data. Visualize, zoom, pan, and color-encode large-scale embedding projections. Interactions enable users to explore millions of points smoothly,



Figure 3: The density mode (right) visualizes the density of the projected points through opacity and contours. When showing multiple categories, the visualization uses order-independent transparency [13] to avoid the effect of blend order.

without needing to pre-sample or pre-render their data, reducing the effort required to explore data at scale.

- F2. Multi-coordinated metadata views.** Combine the scalable embedding, charts, and virtualized table. View coordination removes the need to constantly switch between tools, enabling users to cross-reference points and metadata together.
- F3. Automatic data clustering & labeling.** Group and label related points across multiple resolutions. By surfacing data structure automatically, users do not need to manually group or annotate clusters, which is especially helpful when exploring a new, unfamiliar dataset.
- F4. Density contour rendering.** Summarize points with real-time contour plotting based on data density. Density overlays (Figure 3) help users quickly identify dense or sparse regions without zooming or relying on trial-and-error, streamlining spatial reasoning about the embedding layout.
- F5. Real-time search & nearest neighbors.** Find similar data to any point. Rather than manually scanning the visualization or writing code to find related data, users can quickly locate relevant items in real time.
- F6. Optional in-browser vector computation.** Choose whether to use browser-based WASM or CPU/GPU-based models and UMAP for computing and reducing high-dimensional vectors. Removes the need to install packages or configure environments, allowing users to get started with only a web browser—ideal for onboarding, demos, or lightweight workflows.
- F7. Flexible data imports.** Load data locally or from Hugging Face with one command, using standard formats like CSV, JSON, or Parquet. There is no required schema or prescribed structure. Avoids the overhead of converting data into custom formats, allowing users to work with the data as-is.
- F8. Extensibility.** Use EMBEDDING ATLAS as a web tool, in Python notebooks (Figure 2), as a CLI, or in Streamlit. EMBEDDING ATLAS aims to support major analytical environments, allowing users to stay in their platform of choice, and by exporting selections. Additionally, EMBEDDING ATLAS can be integrated into a larger applications as a component.

3.3 “Quality of Life” Design

While we detail features of the system above, note that many “quality of life” details have been designed to further reduce user friction.

For example, Figure 1 shows EMBEDDING ATLAS using a wine review dataset,¹ imported directly from HuggingFace. While the

¹<https://huggingface.co/datasets/spawn99/wine-reviews>

embedding is computed on the wine review text (description column), each wine also contains metadata such as its origin country and province, its score in points, price, and variety. In EMBEDDING ATLAS, the distribution of each metadata column is automatically visualized in the sidebar (supported data types are numerical, categorical, and multi-value categorical), and users can cross-filter between all the charts and the embedding. For example, in Figure 1 a user has filtered to only show wines from the US, Italy, and France, which highlights 140,732 of 196,630 reviews. Users can also add new charts beyond univariate histograms, such as 2D heatmaps, stacked bar charts, and box plots (see Figure 4 for more). A virtualized data table can also be shown for users who want to browse the data quickly.

Other quality of life improvements include implementing order-independent transparency [13] to avoid the effect of blend order. The automatic cluster labels are also produced with a fast clustering algorithm based on 2D density [22], and labels are positioned using a map-like de-overlapping algorithm that maintains consistency while zooming [2]. Lastly, the metadata charts automatically select scale type based on data and also gracefully handle invalid values like null, nan, and inf—while allowing users to see the number of those values and even select them.

3.4 Implementation

EMBEDDING ATLAS leverages Mosaic [5], which, in combination with DuckDB [19], enables efficient cross-filtering across large data. Mosaic coordinates interactions between visual components, while DuckDB executes analytical queries with high performance directly in the browser via WebAssembly [10], enabling responsive, in-browser data exploration without the need for a backend server. The embedding view is implemented in WebGPU, using a vertex shader to generate quads from data entries, and a fragment shader to create the circle shapes and write necessary information to produce order-independent transparency [13]. The real-time density map is also implemented in WebGPU with a compute kernel for kernel density estimation using Deriche approximation [4], and fragment shaders to render the density levels and contours.

EMBEDDING ATLAS leverages the SentenceTransformers library [21] to compute embeddings from textual data, and the Transformers library [27] for processing image data. Users may specify any embedding model compatible with these libraries. To visualize these high-dimensional embeddings, we use UMAP [15] to project them into 2D coordinates. Alternatively, users may supply pre-computed 2D coordinates.

4 EVALUATION

4.1 Competitive Analysis

Two authors compared the features of EMBEDDING ATLAS to other popular embedding visualizations tools. We exclude many tools that are effectively scatterplots built on plotting libraries (e.g., Vega-Lite [23], Plotly [8]), since these tools immediately suffer from scalability as the data grows into the few thousands. We report on feature comparison in Table 1 and state where a tool fully or partially supports a feature. Note that partial support denotes a tool’s feature has a limitation or is a subset of EMBEDDING ATLAS’s capabilities (e.g., coloring points by 2 categories instead of N).

Notice that most tools focus on handling large scale data (F1), however, these tools solely focus on rendering, and do not visualize other metadata, let alone support cross-filtering at this scale (F2). None of the tools support the full set of data interactions (F3–F5) EMBEDDING ATLAS implements, which help users explore their data rather than solely rendering it. Although tools focus on getting diverse types of data in (F7), few allow users to export results in other developer environments (F8), and almost all require users to use backends or servers for computation (F6).

Table 1: A feature comparison of EMBEDDING ATLAS against other popular embedding visualization tools. Legend: ● denotes a tool fully satisfies the feature, whereas ◐ denotes a tool partially satisfies the feature.

	Embedding Projector (2016)	WizMap (2023)	Nomic Atlas (2023)	Data Map Plot (2024)	Latent Scope (2024)	EMBEDDING ATLAS (2025)
F1. Fast interactive rendering of massive data	◐	●	●	●	●	●
F2. Multi-coordinated metadata views						●
F3. Automatic data clustering & labeling		◐	●		●	●
F4. Density contour rendering		●		●		●
F5. Real-time search & nearest neighbors	●	◐	◐	◐	◐	●
F6. Optional in-browser vector computation	●				◐	●
F7. Flexible data imports	●	◐	●	◐	●	●
F8. Extensibility		◐	◐		◐	●

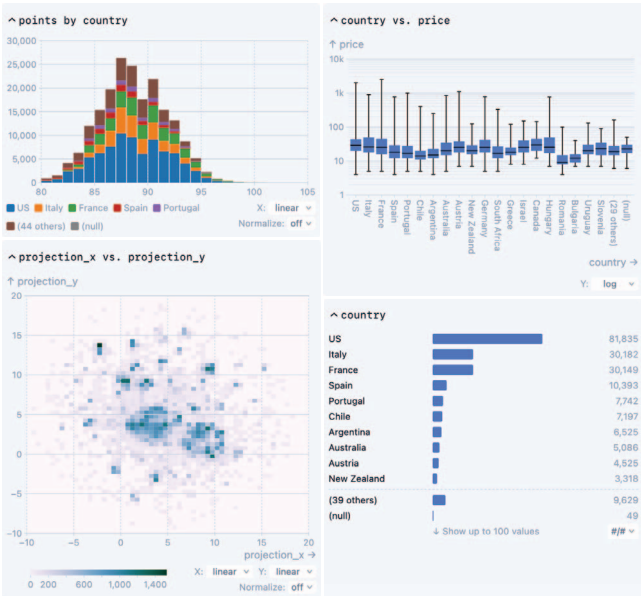


Figure 4: EMBEDDING ATLAS includes multiple distribution visualizations: bar charts, histograms, 2D heatmaps, and box plots. Basic charts can be added via a template-based chart builder. Advanced users can write arbitrary Mosaic specs to create complex charts.

4.2 Rendering Performance

We evaluate the rendering performance of the web-based embedding visualization by its frame rate. Figure 5 shows the results of an experiment with synthetic datasets of variable number of points and categories. The experiment is conducted with a 1600 × 1600 pixel embedding view (for 2x display scale), on an Apple M1 Pro with 32GB RAM, 10 core CPU, and 16 core GPU. EMBEDDING ATLAS can render up to 4 million points at 60fps, and maintain performance at 25fps for 10+ million points. With 1600 × 1600 pixels we did not notice a significant difference in FPS between points and density mode (Figure 3). However, since the density mode uses more involved pixel-space computation, its FPS drops as we increase the number of pixels (e.g., for 5 million points, FPS drops from 46 to 33 if we change to a 3840 × 2160 pixels view).

5 Discussion

Flexible Data Processing with DuckDB and Mosaic EMBEDDING ATLAS supports both in-browser and server-side analysis via DuckDB and Mosaic [5], offering flexibility based on data size and user needs. For small datasets, run EMBEDDING ATLAS

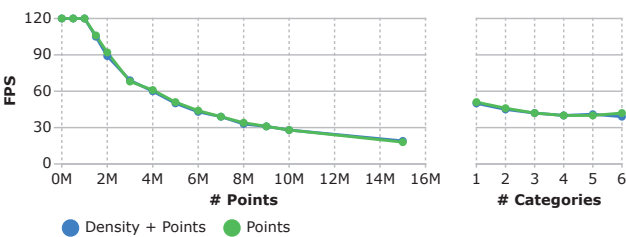


Figure 5: Left: Embedding view frames per second (FPS) over the number of points in the dataset. EMBEDDING ATLAS maintains above 60fps until 4 million points. Right: FPS over the number of categories for 5 million points.

directly in the browser for a fast and installation-free experience. For larger datasets, EMBEDDING ATLAS connects to a server-side DuckDB instance to leverage more RAM and processing power.

Multiple Deployment Options For lighter use cases or quick exploration, EMBEDDING ATLAS has a web-based endpoint that allows users to upload and visualize datasets directly in the browser with no installation required. For more advanced users, a command-line interface, Jupyter, and Streamlit widgets enable easy scripting and automation for complex workflows. Additionally, EMBEDDING ATLAS can be embedded as a component into into existing tools, dashboards, or research platforms.

6 LIMITATIONS AND FUTURE WORK

Embedding Comparison Comparing different datasets or dataset versions is often useful. While EMBEDDING ATLAS does not yet provide first-class comparison support, users can simulate it by adding auxiliary columns (e.g., dataset labels or version numbers) and using filtering or coloring. This approach has limitations (e.g., no side-by-side view), and first-class support would greatly enhance the comparison experience.

Towards a Dashboard-like Experience EMBEDDING ATLAS currently offers a single layout of the embedding view, charts, and the table. If there is no data specified for the embedding view, the tool stills shows the table and distribution charts; however, as we attempt to cover more use cases, this fixed layout becomes a limitation. In future versions, we would like to create a dashboard-like experience where users may freely design their own layout that best suit their analytical needs.

User Evaluation A user study is beyond the scope of this short paper. However, we plan to deploy the system to real-world data analysts, report its usage in practical settings, and collect feedback to guide future improvements. Since the tool is also open-sourced, we hope the community will guide future directions.

ACKNOWLEDGMENTS

We thank our colleagues at Apple, with a special mention to Yannick Assogba, for giving feedback on early drafts of this work.

REFERENCES

- [1] D. Atzberger, T. Cech, W. Scheibel, J. Döllner, M. Behrisch, and T. Schreck. A large-scale sensitivity analysis on latent embeddings and dimensionality reductions for text spatializations. *IEEE Transactions on Visualization and Computer Graphics*, 31(1):305–315, 2025. doi: 10.1109/TVCG.2024.3456308 1
- [2] K. Been, E. Daiches, and C. Yap. Dynamic map labeling. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):773–780, 2006. doi: 10.1109/TVCG.2006.136 3
- [3] N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, S. Kravec, Z. Hatfield-Dodds, R. Lasenby, D. Drain, C. Chen, et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022. 2
- [4] J. Heer. Fast & accurate gaussian kernel density estimation. In *IEEE Visualization Conference*, pp. 11–15. IEEE, 2021. doi: 10.1109/VIS49827.2021.9623323 3
- [5] J. Heer and D. Moritz. Mosaic: An architecture for scalable & interoperable data views. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):436–446, 2024. doi: 10.1109/TVCG.2023.3327189 2, 3, 4
- [6] F. Hohman, M. Kahng, R. Pienta, and D. H. Chau. Visual analytics in deep learning: An interrogative survey for the next frontiers. *IEEE Transactions on Visualization and Computer Graphics*, 25(8):2674–2693, 2018. doi: 10.1109/TVCG.2018.2843369 2
- [7] Z. Huang, D. Witschard, K. Kucher, and A. Kerren. VA + Embeddings STAR: A state-of-the-art report on the use of embeddings in visual analytics. *Computer Graphics Forum*, 42(3):539–571, 2023. doi: 10.1111/cgf.14859 1
- [8] P. T. Inc. Collaborative data science. <https://plot.ly>, 2015. Accessed: 2025-04-17. 3
- [9] I. Johnson. Latent Scope. <https://github.com/enjalot/latent-scope>, 2024. Accessed: 2025-04-17. 2
- [10] A. Kohn, D. Moritz, M. Raasveldt, H. Mühleisen, and T. Neumann. DuckDB-wasm: Fast analytical processing for the web. *Proceedings of the VLDB Endowment*, 15(12):3574–3577, 2022. doi: 10.14778/3554821.3554847 1, 3
- [11] A. Maćkiewicz and W. Ratajczak. Principal components analysis (PCA). *Computers & Geosciences*, 19(3):303–342, 1993. 1, 2
- [12] T. Manz, F. Lekschas, E. Greene, G. Finak, and N. Gehlenborg. A general framework for comparing embedding visualizations across class-label hierarchies. *IEEE Transactions on Visualization and Computer Graphics*, 31(1):283–293, 2025. doi: 10.1109/TVCG.2024.3456370 1
- [13] M. McGuire and L. Bavoil. Weighted blended order-independent transparency. *Journal of Computer Graphics Techniques (JCGT)*, 2(2):122–141, 2013. 3
- [14] L. McInnes. DataMapPlot: Creating beautiful plots of data maps. <https://datamapplot.readthedocs.io/en/latest/>, 2023. Accessed: 2025-04-17. 2
- [15] L. McInnes, J. Healy, and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2020. 1, 2, 3
- [16] A. Ng et al. Sparse autoencoder. *CS294A Lecture notes*, 72(2011):1–19, 2011. 2
- [17] Nomic AI. Nomic Atlas. <https://atlas.nomic.ai/>, 2022. Accessed: 2025-04-17. 1, 2
- [18] B. A. Olshausen and D. J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, 1996. doi: 10.1038/381607a0 2
- [19] M. Raasveldt and H. Mühleisen. DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data*, pp. 1981–1984, 2019. doi: 10.1145/3299869.3320212 1, 3
- [20] M. Ranzato, Y.-L. Boureau, Y. Cun, et al. Sparse feature learning for deep belief networks. *Advances in Neural Information Processing Systems*, 20, 2007. 2
- [21] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019. 3
- [22] D. Ren, F. Hohman, and D. Moritz. A scalable approach to clustering embedding projections. *arXiv preprint arXiv:2504.07285*, 2025. 2, 3
- [23] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-Lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23:341–350, 2016. doi: 10.1109/TVCG.2016.2599030 3
- [24] D. Smilkov, N. Thorat, C. Nicholson, E. Reif, F. B. Viégas, and M. Wattenberg. Embedding Projector: Interactive visualization and interpretation of embeddings. *arXiv preprint arXiv:1611.05469*, 2016. 2
- [25] L. Van der Maaten and G. Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(11), 2008. 1, 2
- [26] Z. J. Wang, F. Hohman, and D. H. Chau. WizMap: Scalable interactive visualization for exploring large machine learning embeddings. In *Demo, Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. doi: 10.18653/v1/2023.acl-demo.50 1, 2
- [27] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush. HuggingFace’s Transformers: State-of-the-art natural language processing, 2020. 3